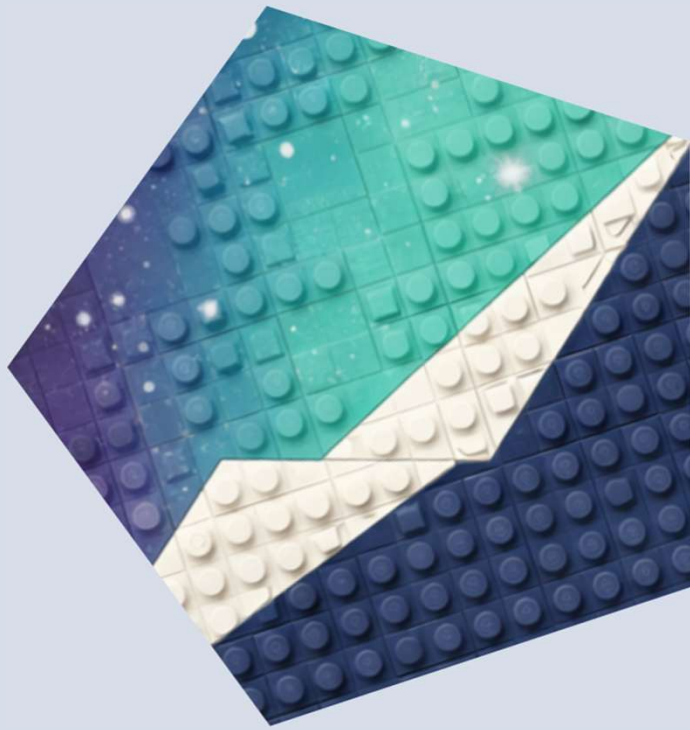




nordic summit

Billund | 21st-22nd September 2026

Multi-Agent Orchestration in Copilot Studio

Jennifer Krause · Holger Imbery · NTT DATA

Pick the harness first: which building block works where, and why.





Gold Sponsors



Silver Sponsors



Bronze Sponsors

Who are we?
We build these systems every day.

No feature hype –
just things that work for us in projects

Jennifer Krause
Consultant · agentic systems, AI automation
NTT DATA

Holger Imbery
Principal Architect · agentic systems
NTT DATA



Find our slides later in the App or in our repo: <https://github.com/instruo365/talks>



1. Always start with your business process
2. Do you really need multi-agent orchestration? (specialization vs. reusability)
3. A deeper look at the Copilot Studio harnesses and available building blocks
 - Connecting agents and tools: MCP and A2A
 - Child agents specialize. Skills package behavior.
4. Select your Harness based on what you need
5. Multi-Agent Patterns and four blueprints
6. Practical example: Invoice Processing
7. Key Takeaways, common mistakes, and next steps

Start with the process. Earn the agent.

- 1 **OUTCOME:** What measurable result should improve?
- 2 **PROCESS:** Is the actual path mapped, including exceptions?
- 3 **AI FIT:** Does the work require interpretation or a dynamic route?
- 4 **ARCHITECTURE:** Does a second agent create a real domain, owner, or control boundary?

Default: fix the process, automate the rules, then add agentic complexity only where it earns its place.



Start with one Agent.

Split when routing degrades or the work reveals a stable boundary.

ONE AGENT

Clear purpose
Shared context
One owner

CHILD AGENTS

Focused specialists
inside one cohesive
solution

CONNECTED AGENTS

Independent lifecycle,
settings, publication,
ownership, or reuse

DOMAIN

Different
knowledge,
tools, or expertise

CONTROL

Different
permissions,
risk, or governance

OWNERSHIP

Independent
lifecycle,
team, or reuse

30–40 choices of action

Microsoft rule of thumb: review routing precision, improve descriptions first,
then split if differentiation still degrades.

SPLIT WHEN

- ✓ Routing precision degrades in evaluations
- ✓ A specialist has one clear responsibility
- ✓ Knowledge or tools are meaningfully distinct
- ✓ Independent settings, lifecycle, or reuse are required

DO NOT SPLIT AUTOMATICALLY

- × The number 30 is reached
- × The instruction file is getting long
- × The organization chart has several teams
- × “More agents” sounds more advanced

Design Principles:

30–40 actions is a *review point*. Evaluation evidence and meaningful boundaries decide the split.
Transfers between harnesses are not possible — you need to consider your choice before selecting the appropriate harness

Three harnesses. Two runtimes for this talk.

The harness is the runtime between your design and the model.
It determines how work is executed, what is available, and how usage is billed.

GITHUB COPILOT Harness

GOAL-DRIVEN

BEST FOR

Reasoning-heavy, multi-step work

HOW IT WORKS

Plans from a goal
Chooses tools and adapts
Retries or finds another path

Native files, skills, memory, sandbox

Billing: Copilot Credits, consumption-based

STANDARD Harness

PATH-DRIVEN

BEST FOR

Structured, repeatable work

HOW IT WORKS

Follows topics, triggers,
rules, branches, and paths
Predictable for known requests

Agents and agent flows

Billing: Copilot Studio capacity / licensing

COPILOT CHAT Harness

KNOWLEDGE-DRIVEN

BEST FOR

Extend Microsoft 365 Copilot Chat

HOW IT WORKS

Connects enterprise knowledge
to the existing Chat experience
Publishes to internal teams

Grounded answers inside Microsoft 365

Billing: consumption or included with M365 Copilot

FROM HERE ON: Standard harness + GitHub Copilot harness

The harness defines your building-block palette.

Some capabilities bridge both runtimes. Others commit the design to one harness.

STANDARD HARNESS ONLY

2 capability groups

CHILD AGENTS

Lightweight specialists inside the parent agent

EXTERNAL AGENT CONNECTIONS

A2A; Foundry, Fabric, and Agents SDK options*

AVAILABLE IN BOTH

3 shared bridges

COPILOT STUDIO CONNECTED AGENTS

Delegate to another Copilot Studio agent

MCP SERVERS

Expose standardized tools and resources

WORKFLOWS + CONNECTORS

Deterministic automation and system actions

GITHUB COPILOT HARNESS ONLY

3 capability groups

SKILLS

Reusable packages of instructions, scripts, and resources

MEMORY · PREVIEW

User-specific context across interactions

NATIVE FILE WORK + SANDBOX

Create, edit, and reason over Office files and PDFs

ARCHITECTURE RULE: If a required capability sits in an outer column, the harness decision is already constrained.

* Foundry, Fabric, and Microsoft 365 Agents SDK connections are documented as preview. Verify current tenant availability.

Connecting Tools and Agents: MCP and A2A

MCP connects tools, while A2A connects agents. Both are complementary, not alternatives

MCP · MODEL CONTEXT PROTOCOL

AGENT → TOOLS + RESOURCES

THE CONTRACT

An MCP server publishes named tools and resources, including their descriptions, inputs, and outputs.

USE WHEN

- ✓ Need functions or data from a service
- ✓ Want standardized tool discovery
- ✓ No independent agent reasoning required

STANDARD + GITHUB COPILOT HARNESS

A2A · AGENT2AGENT*

AGENT → ANOTHER AGENT

THE CONTRACT

An A2A-enabled agent receives a delegated task, uses its own reasoning or workflow, and returns an agent response.

USE WHEN

- ✓ The collaborator is already an agent
- ✓ It owns domain-specific reasoning or workflows
- ✓ The interaction can be multiturn

STANDARD HARNESS

DECISION TEST: Do you need a capability to call, or an autonomous collaborator to delegate to?

*A2A is documented for the Standard harness today; and in Preview for Github Copilot harness.

Child agents specialize. Skills package behavior.

They can both make a solution modular, but they solve different design problems and run on different harnesses.

CHILD AGENT · STANDARD HARNESS

A specialist inside the parent solution

WHAT IT OWNS

Its own instructions, tools, knowledge, orchestration, and optional inputs and outputs.

USE WHEN

- ✓ The parent should delegate a bounded task
- ✓ The specialist needs dedicated tools or knowledge
- ✓ You need explicit invocation, inputs, or outputs

WHO SHOULD DO IT?

SKILL · GITHUB COPILOT HARNESS

A reusable task-specific capability

WHAT IT PACKAGES

Instructions in Markdown, with optional scripts, templates, reference material, and other resources.

USE WHEN

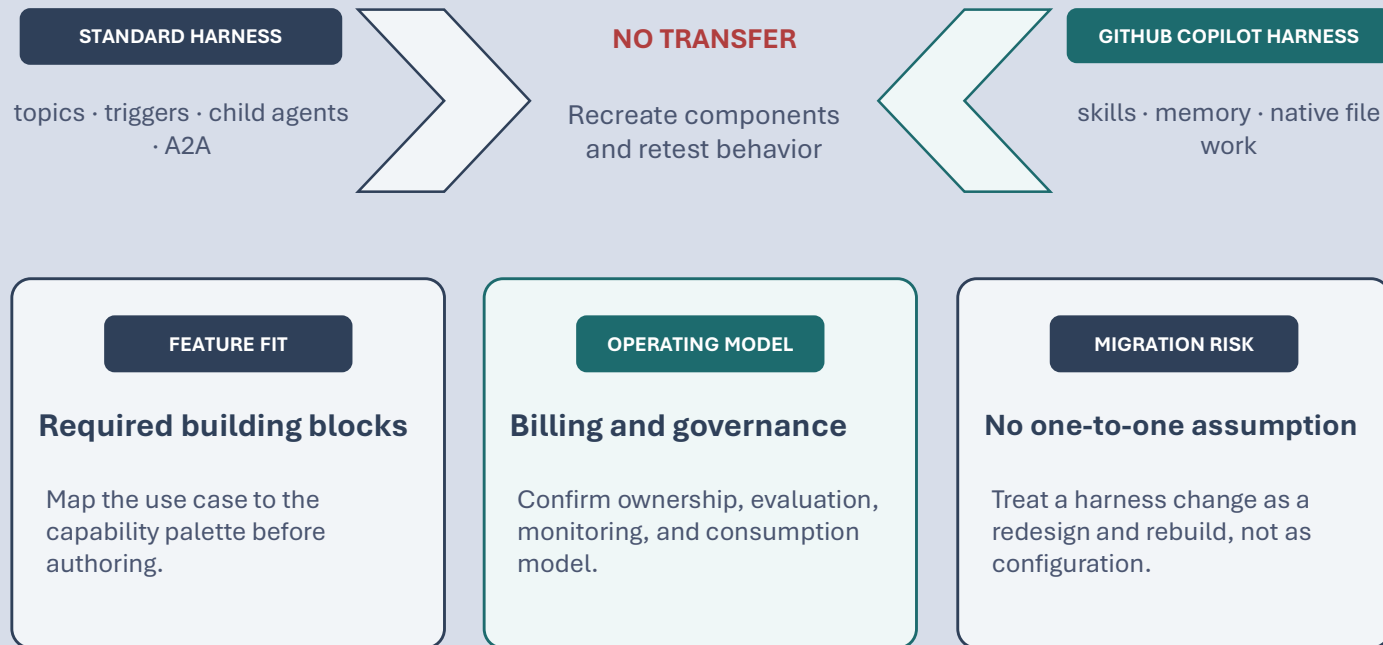
- ✓ The same procedure should be reused
- ✓ You want behavior without another agent hop
- ✓ The agent should activate a role or method on demand

HOW SHOULD IT BE DONE?

DECISION TEST: Do you need another reasoner, or a reusable way of working?

Choose the harness before you build.

The harness is not a runtime switch. Moving later means rebuilding the agent on a different foundation.



DESIGN RULE: Prototype on the runtime you can defend in production.

Overview: Harness Selection

Four questions to help you select the right harness for your use case.



STANDARD HARNESS

No

- **Child Agents**
- Agent-to-Agent (A2A, GA)
- External agents (Foundry, Fabric, Agents SDK)

EITHER HARNESS

One Agent

Multi-Agents

- **Connected agents (Copilot Studio)**
- MCP as a tool
- Workflows / Deterministic Automation

GITHUB COPILOT HARNESS (\$\$)

Yes

- **Skills** (Sandbox & File Generation)
- Memory (Preview)

Patterns for Multi-Agent Systems



Pattern 01 · Orchestrator with Specialist Agents.

One Agent owns the Conversation; the remaining Agents execute Tasks.



WHEN IT FITS

One dominant business use case. A clear owner of the experience.
Most calls stay inside one business domain.

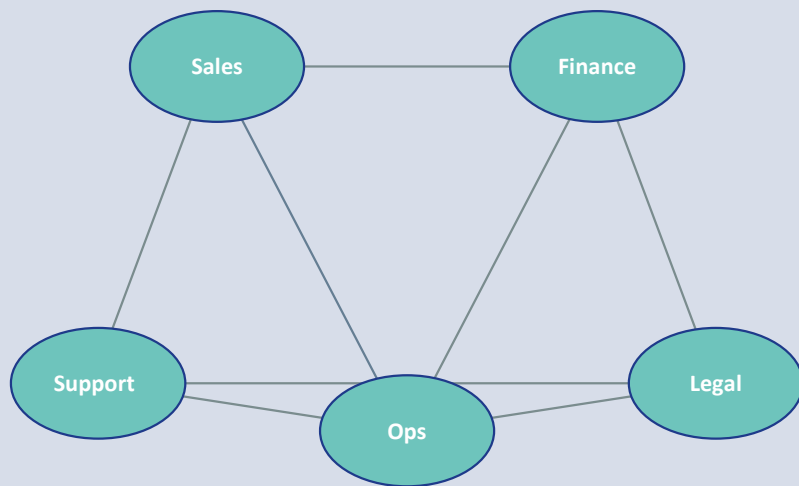
WATCH OUT

The Orchestrator Agent becomes a bottleneck as the number of Specialist Agents grow. Keep its instructions narrow and focus on detailed Child Agent descriptions instead — The Orchestrator Agent decides, it does not execute.

The user only communicates with the Orchestrator Agent. It interprets intent, plans steps, and dispatches to Child or Connected Agents.

Pattern 02 · Specialist network.

Peers hand off when a task crosses a business domain.



WHEN IT FITS

Multiple business domains of comparable weight. Strong independent ownership per agent. Workflows that routinely cross domain lines.

WATCH OUT

Harder to reason about. Publish clear agent cards and keep the number of peers small, or nobody can trace a request end to end.

Each agent is a domain expert and knows which peers to invoke for work outside its own domain.

Four blueprints, four different shapes. Each one is a different orchestration pattern.

IT INCIDENT TRIAGE — Orchestrator-Specialist

Classify the ticket, delegate to a runbook specialist and a ticket writer. Structured outputs between agents.

CONTRACT REVIEW — Specialist Network

Legal, finance and risk reviewers run at once over scoped policy libraries; an orchestrator merges the memo.

FIELD-SERVICE DISPATCH — One Autonomous Trigger

A Dataverse row wakes the agent; it checks stock, reschedules, and notifies. No user in the loop.

EMPLOYEE ONBOARDING — Hierarchical, Multiple Triggers

Long-running, cross-departmental. An orchestrator reads state and delegates; specialists write only their own elements. Wait – restart – execute – with a central state store.

Practical Example: Invoice Processing.

The process is not the agent — the process is the state stored in a table.

WHY A “CHAT” CANNOT HOLD IT

An invoice comes in on Tuesday, waits four days for approval, is sent back because the PO number is missing, returns on Monday, and is booked two weeks later. A Copilot Studio chat stops after 30 minutes without activity.

THE STATE IS THE MAIN ELEMENT

The agent works like a helper: it takes an item, does one small job, writes down what it did, and moves the item to the next step. The process itself lives in Dataverse.

Received → Classified → Extracted → Validated → Matched → PendingApproval → Approved → Posted → Closed
plus, the two states for real life: NeedsHuman and Failed.

Built entirely on the standard harness — Copilot Studio, agent flows, Dataverse, connectors.
No extra middleware, no second platform, no separate system to run.

A thin orchestrator, nine small specialists. The table is the shared memory.

ORCHESTRATOR AGENT — reads the case, routes by state, and writes the result back. Three tools: `GetNextCases` · `TransitionCase` · child agents

CLASSIFIER

Invoice, credit note, dunning letter? Cheap and fast — keeps expensive extraction off documents that never needed it.

TWO EXTRACTION AGENTS

Scanned via OCR, digital via text layer — one shared JSON schema, and per-field confidence, not just values.

VALIDATOR & MATCHER

Arithmetic, tax, IBAN, duplicates; PO and goods-receipt matching. Deviation lists, never decisions.

APPROVAL AGENT

“Waits” means: write `PendingApproval`, set `NextActionAt` for the reminder, and stop. Nothing stays in memory.

POSTING & ALERTING

Backend posting with write-before, confirm-after. SLAs, reminders and escalations recorded in the table.

SUPPLIER INQUIRY

Missing PO or tax ID? Mail the sender, move to `WaitingForSupplier`, set a deadline, park the case cleanly.

Each specialist gets only the case ID and reads what it needs. Deterministic logic in flows, judgement in agents, truth in Dataverse.

Six things people get wrong. The first one costs the most.

"A HARNESS IS JUST A SETTING." — THE COSTLY ONE

It is a runtime. You cannot switch one — you rebuild the agent.

"SKILLS REPLACE CHILD AGENTS."

They absorb many use cases, but no skill gives you a trigger or a typed input.

"A SKILL CAN CALL AN API."

Skill code has no outbound network path. Everything leaves through a tool.

"MCP AND A2A COMPETE."

MCP connects an agent to tools. A2A connects an agent to an agent. Adopt both.

"CONNECTED AGENTS FEATURE CAN CONNECT ANY AGENT"

On the Github Copilot harness, you can currently only connect agents built in Copilot Studio (GA)

"MORE AGENTS MEAN MORE CAPABILITY."

Keep a single Agent when your actions are below the threshold of 30-40. Simpler maintenance and lower orchestration overhead and latency.

Neither harness wins. Choose based on your use-case

Standard: predictable paths, existing licensing.
GitHub Copilot: reasoning, skills, files, recovery.
You cannot switch later — design for each deliberately.



Take four things back with you.

Then pick one use case and start.

1 · USE CASE FIRST, THEN RUNTIME

Verify the use case before anything else, then choose the appropriate harness, because it is the one decision you cannot undo. Every other architectural decision is bound to your harness.

2 · JUSTIFY ADDING ANY NEW (CHILD) AGENT

One agent and clearer instructions first. For fewer than 30–40 actions, a second agent would only add latency and governance, not additional capability.

3 · TRUTH IN A TABLE, NOT IN A “CHAT”

Anything that outlives a session needs a documented state. The agent decides, flows execute, Dataverse remembers.

4 · EVERY USE CASE NEEDS A HUMAN OWNER

One domain, one named human owner, one identity per agent. Control boundaries must be visible and documented — deterministic logic in flows, judgement in agents.

Opening the Maker Portal is always the last step.

Thank you!



Link to Feedback Form





THANK YOU